

Forms & Form Validation — ফর্ম তৈরি, ডাটা নেওয়া এবং Validation করা

আধুনিক ওয়েব অ্যাপ্লিকেশন বা সিঙ্গেল পেজ অ্যাপে ইউজার ইন্টারঅ্যাকশনের সবচেয়ে গুরুত্বপূর্ণ মাধ্যম হলো **Form**। ব্যবহারকারীর কাছ থেকে লগইন, রেজিস্ট্রেশন, পেমেন্ট বা যেকোনো ইনপুট সুরক্ষিতভাবে গ্রহণ ও যাচাই করার জন্য জাভাস্ক্রিপ্ট ফর্ম হ্যান্ডলিং ও ভ্যালিডেশন অত্যন্ত অপরিহার্য।

Form Elements → preventDefault() → Constraint Validation / Regex → FormData API → Clean Payload

Frontend ও Backend উভয়ের নিরাপত্তায় নির্ভুল ও ডায়নামিক ক্লায়েন্ট-সাইড ভ্যালিডেশন প্রথম প্রাচীর হিসেবে কাজ করে। 🔥

PART 01 — FORM FUNDAMENTALS, STRUCTURE & FORM ELEMENTS (17.1 – 17.8)

17.1 Form কী?

Form হলো এমন একটি UI অংশ যেখানে user information input করে।

💡 REAL-LIFE EXAMPLE

তুমি কোনো website-এ registration করছো:

```
Name:      Shariar
Email:     shariar@example.com
Password:  *****
Phone:     017XXXXXXX

[ Register ]
```

এই পুরো অংশটাই একটি **Form**।

HTML:

17.1 HTML

MARKUP

```
<form>
  <input type="text" />
  <input type="email" />
  <input type="password" />

  <button type="submit">Register</button>
</form>
```

17.2 Basic Form Structure

17.2 HTML

MARKUP

```
<form id="registrationForm">
  <label for="name">Name</label>
  <input type="text" id="name" />

  <label for="email">Email</label>
  <input type="email" id="email" />

  <button type="submit">Register</button>
</form>
```

JavaScript:

17.2 JAVASCRIPT

RUN

```
const form = document.querySelector("#registrationForm");

form.addEventListener("submit", function (event) {
  event.preventDefault();

  console.log("Form submitted!");
});
```

Form submitted!

⚙️ এখানে কী হলো?

17.2 JAVASCRIPT

RUN

```
form.addEventListener("submit", ...)
```

Form submit হলে function চলবে।

আর:

17.2 JAVASCRIPT

RUN

```
event.preventDefault();
```

browser-এর default form submission বন্ধ করবে।

17.3 কেন `preventDefault()` ব্যবহার করি?

Normally form submit করলে browser page reload করতে পারে।

17.3 JAVASCRIPT

RUN

```
form.addEventListener("submit", function (event) {
  event.preventDefault();
});
```

এতে browser-এর default behavior বন্ধ হয়।

💡 REAL-LIFE EXAMPLE

Login form submit করলে তুমি চাও:

```
User clicks Login
↓
JavaScript data নেয়
↓
Validation করে
↓
API-তে data পাঠায়
↓
Login success
```

Page reload করানো দরকার নেই।

17.4 Input থেকে Value নেওয়া

17.4 HTML

MARKUP

```
<input type="text" id="username" />

<button id="btn">Get Name</button>
```

17.4 JAVASCRIPT

RUN

```
const username = document.querySelector("#username");
const button = document.querySelector("#btn");

button.addEventListener("click", function () {
  console.log(username.value);
});
```

User যদি লেখে:

TEXT

Shariar

TEXT

Shariar

★ গুরুত্বপূর্ণ

Input-এর value পেতে:

17.4 JAVASCRIPT

RUN

```
input.value;
```

17.5 Form-এর সব Input Access করা

17.5 HTML

MARKUP

```
<form id="myForm">
  <input type="text" name="username" />

  <input type="email" name="email" />

  <input type="password" name="password" />

  <button type="submit">Submit</button>
</form>
```

JavaScript:

17.5 JAVASCRIPT

RUN

```
const form = document.querySelector("#myForm");

console.log(form.elements);
```

`form.elements` form-এর controls-এর collection দেয়।

17.6 form.elements` দিয়ে নির্দিষ্ট Input

17.6 JAVASCRIPT

RUN

```
const form = document.querySelector("#myForm");

console.log(form.elements.username.value);
console.log(form.elements.email.value);
console.log(form.elements.password.value);
```

CONSOLE OUTPUT

```
Shariar
shariar@gmail.com
12345678
```

এখানে `name` attribute গুরুত্বপূর্ণ।

17.6 HTML

MARKUP

```
<input name="username" />
```

17.7 Label কেন ব্যবহার করি?

ভালো form-এর জন্য:

17.7 HTML

MARKUP

```
<label for="email">Email</label>

<input type="email" id="email" />
```

`for="email"` → input-এর `id="email"` -এর সাথে সম্পর্ক তৈরি করে।

User label-এ click করলেও input focus হবে।

17.8 Different Input Types

Form-এ বিভিন্ন ধরনের input আছে।

17.8 HTML

MARKUP

```
<input type="text" />

<input type="email" />

<input type="password" />

<input type="number" />

<input type="tel" />

<input type="date" />

<input type="time" />

<input type="url" />

<input type="checkbox" />

<input type="radio" />

<input type="file" />

<input type="color" />

<input type="range" />
```

JavaScript দিয়ে এগুলোর value/behavior handle করা যায়।

17.9 required

User যেন field empty রেখে submit করতে না পারে:

17.9 HTML

MARKUP

```
<input type="text" name="username" required />
```

User empty রেখে submit করলে browser validation message দেখাবে।

17.10 minlength এবং **maxlength**

17.10 HTML

MARKUP

```
<input type="text" minlength="3" maxlength="20" />
```

এর মানে:

```
Minimum → 3 characters  
Maximum → 20 characters
```

17.11 min এবং **max**

Number-এর ক্ষেত্রে:

17.11 HTML

MARKUP

```
<input type="number" min="18" max="60" />
```

Allowed range:

```
18 → 60
```

17.12 pattern

Custom pattern তৈরি করা যায়।

17.12 HTML

MARKUP

```
<input type="text" pattern="[A-Za-z]+" />
```

এখানে শুধুমাত্র English alphabet allow করা হচ্ছে।

17.13 autocomplete

Browser যেন আগের information suggest করতে পারে:

17.13 HTML

MARKUP

```
<input type="email" autocomplete="email" />
```

Password:

17.13 HTML

MARKUP

```
<input type="password" autocomplete="current-password" />
```

Registration-এর নতুন password:

17.13 HTML

MARKUP

```
<input type="password" autocomplete="new-password" />
```

17.14 novalidate

Form-এর browser validation বন্ধ করতে:

17.14 HTML

MARKUP

```
<form novalidate></form>
```

এটি ব্যবহার করলে validation JavaScript দিয়ে নিজে করতে পারো।

17.15 HTML Constraint Validation

HTML নিজেই অনেক validation করতে পারে।

17.15 HTML

MARKUP

```
<form>
  <input type="email" required />

  <input type="number" min="18" max="60" />

  <button type="submit">Submit</button>
</form>
```

Browser automatically check করবে:

```
required
email format
min
max
pattern
minlength
maxlength
```

17.16 checkValidity()

কোনো form বা input valid কিনা check করতে:

17.16 JAVASCRIPT

RUN

```
const email = document.querySelector("#email");

console.log(email.checkValidity());
```

Valid হলে:

TEXT

true

Invalid হলে:

TEXT

false

17.17 Example

17.17 HTML

MARKUP

```
<input id="email" type="email" required />
```

JavaScript:

17.17 JAVASCRIPT

RUN

```
const email = document.querySelector("#email");  
  
console.log(email.checkValidity());
```

Empty হলে:

TEXT

false

Valid email দিলে:

TEXT

true

17.18 reportValidity()

Browser-এর built-in validation message দেখাতে:

17.18 JAVASCRIPT

RUN

```
email.reportValidity();
```

যদি email invalid হয় browser message দেখাবে।

17.19 validity` Object

JavaScript-এর সবচেয়ে গুরুত্বপূর্ণ form validation concepts-এর একটি।

17.19 JAVASCRIPT

RUN

```
console.log(email.validity);
```

এখানে অনেক property পাওয়া যায়।

যেমন:

17.19 JAVASCRIPT

RUN

```
email.validity.valid;
```

17.20 Important Validity Properties

Property	Meaning
valueMissing	required field empty
typeMismatch	email/url type ভুল
patternMismatch	pattern match করেনি
tooLong	maximum length ছাড়িয়েছে
tooShort	minimum length-এর নিচে
rangeUnderflow	min-এর নিচে
rangeOverflow	max-এর উপরে
stepMismatch	step-এর সাথে match করছে না
badInput	browser input parse করতে পারছে না
customError	custom validation error
valid	সব validation ঠিক

17.21 Example

17.21 HTML

MARKUP

```
<input id="email" type="email" required />
```

JavaScript:

17.21 JAVASCRIPT

RUN

```
const email = document.querySelector("#email");

console.log(email.validity.valueMissing);
console.log(email.validity.typeMismatch);
console.log(email.validity.valid);
```

Empty থাকলে:

TEXT

```
true
false
false
```

17.22 validationMessage

Browser-এর validation message পাওয়া যায়:

17.22 JAVASCRIPT

RUN

```
console.log(email.validationMessage);
```

যেমন:

TEXT

Please fill out this field.

অথবা email ভুল হলে browser অনুযায়ী message আসতে পারে।

17.23 setCustomValidity()

নিজের custom error message তৈরি করতে পারো।

17.23 JAVASCRIPT

RUN

```
const username = document.querySelector("#username");

if (username.value.length < 5) {
  username.setCustomValidity("Username must be at least 5 characters.");
} else {
  username.setCustomValidity("");
}
```

★ গুরুত্বপূর্ণ

Error remove করতে:

17.23 JAVASCRIPT

RUN

```
setCustomValidity("");
```

17.24 Complete Custom Validation

17.24 HTML

MARKUP

```
<input id="username" type="text" />

<button id="check">Check</button>
```

17.24 JAVASCRIPT

RUN

```
const username = document.querySelector("#username");
const button = document.querySelector("#check");

button.addEventListener("click", function () {
  if (username.value.trim().length < 5) {
    username.setCustomValidity("Username must be at least 5 characters.");

    username.reportValidity();
  } else {
    username.setCustomValidity("");

    console.log("Valid username");
  }
});
```

PART 03 — JAVASCRIPT CUSTOM VALIDATION ALGORITHMS & REGEX (17.25 – 17.31)

17.25 trim() কেন ব্যবহার করি?

User accidentally space দিতে পারে।

17.25 JAVASCRIPT

RUN

```
const name = "  Shariar  ";

console.log(name.trim());
```

TEXT

Shariar

Form validation-৯:

17.25 JAVASCRIPT

RUN

```
const name = input.value.trim();
```

এটি খুব usefull

17.26 Basic Name Validation

17.26 HTML

MARKUP

```
<input id="name" type="text" />

<button id="submitBtn">Submit</button>
```

17.26 JAVASCRIPT

RUN

```
const nameInput = document.querySelector("#name");
const submitBtn = document.querySelector("#submitBtn");

submitBtn.addEventListener("click", function () {
  const name = nameInput.value.trim();

  if (name === "") {
    console.log("Name is required.");
  } else {
    console.log("Name:", name);
  }
});
```

17.27 Email Validation

HTML:

17.27 HTML

MARKUP

```
<input id="email" type="email" />
```

JavaScript:

17.27 JAVASCRIPT

RUN

```
const email = document.querySelector("#email");

if (!email.checkValidity()) {
  console.log("Invalid email");
} else {
  console.log("Valid email");
}
```

17.28 Password Validation

ধরি password minimum 8 characters হতে হবে।

17.28 JAVASCRIPT

RUN

```
const password = "123456";

if (password.length < 8) {
  console.log("Password must contain at least 8 characters.");
} else {
  console.log("Password is valid.");
}
```

```
Password must contain at least 8 characters.
```

17.29 Confirm Password

Real-life registration form-৯:

```
Password:
Confirm Password:
```

JavaScript:

17.29 JAVASCRIPT

RUN

```
const password = "Shariar123";
const confirmPassword = "Shariar123";

if (password !== confirmPassword) {
  console.log("Passwords do not match.");
} else {
  console.log("Password matched.");
}
```

```
Password matched.
```

17.30 Phone Number Validation

ধরি Bangladesh-এর 11-digit phone number check করব।

17.30 JAVASCRIPT

RUN

```
const phone = "01712345678";  
  
const pattern = /^01[3-9]\d{8}$/;  
  
console.log(pattern.test(phone));
```

TEXT

true

ভুল হলে:

17.30 JAVASCRIPT

RUN

```
const phone = "123456";  
  
console.log(pattern.test(phone));
```

TEXT

false

17.31 Regex দিয়ে Email Validation

17.31 JAVASCRIPT

RUN

```
const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;  
  
const email = "shariar@gmail.com";  
  
console.log(emailPattern.test(email));
```

TEXT

true

⚠ তবে production application-এ email validation-এর জন্য শুধু regex-এর উপর নির্ভর করা উচিত নয়।

17.32 Form Submit Handling

এখন একটি complete example দেখি।

HTML

17.32 HTML

MARKUP

```

<form id="signupForm">
  <label>Name</label>
  <input type="text" id="name" required />

  <label>Email</label>
  <input type="email" id="email" required />

  <label>Password</label>
  <input type="password" id="password" minlength="8" required />

  <button type="submit">Sign Up</button>
</form>

```

JavaScript

17.32 JAVASCRIPT

RUN

```

const form = document.querySelector("#signupForm");

form.addEventListener("submit", function (event) {
  event.preventDefault();

  if (!form.checkValidity()) {
    form.reportValidity();
    return;
  }

  const name = document.querySelector("#name").value.trim();

  const email = document.querySelector("#email").value.trim();

  console.log("Name:", name);
  console.log("Email:", email);
});

```

Flow

```

Submit
  ↓
preventDefault()
  ↓
checkValidity()
  ↓
Invalid?
├── Yes → reportValidity()
└── No
    ↓
  Read values
    ↓
  Process / API

```

17.33 FormData

Form-এর data একসাথে collect করার জন্য `FormData` খুব usefull

HTML:

17.33 HTML

MARKUP

```
<form id="loginForm">
  <input type="email" name="email" />

  <input type="password" name="password" />

  <button type="submit">Login</button>
</form>
```

JavaScript:

17.33 JAVASCRIPT

RUN

```
const form = document.querySelector("#loginForm");

form.addEventListener("submit", function (event) {
  event.preventDefault();

  const formData = new FormData(form);

  console.log(formData.get("email"));
  console.log(formData.get("password"));
});
```

17.34 FormData.get()

17.34 JAVASCRIPT

RUN

```
formData.get("email");
```

একটি field-এর value দেয়।

17.35 FormData.has()

17.35 JAVASCRIPT

RUN

```
console.log(formData.has("email"));
```

TEXT

true

17.36 FormData.set()

17.36 JAVASCRIPT

RUN

```
formData.set("email", "newemail@gmail.com");
```

Existing value replace করবে।

17.37 FormData.append()

17.37 JAVASCRIPT

RUN

```
formData.append("role", "student");
```

নতুন data যোগ করবে।

17.38 FormData.delete()

17.38 JAVASCRIPT

RUN

```
formData.delete("password");
```

Field remove করবে।

17.39 FormData-এর সব Data দেখা

17.39 JAVASCRIPT

RUN

```
for (const [key, value] of formData.entries()) {  
  console.log(key, value);  
}
```

```
email shariar@gmail.com  
password 12345678
```

17.40 FormData → Object

অনেক সময় API-তে JSON পাঠানোর জন্য object দরকার হয়।

17.40 JAVASCRIPT

RUN

```
const data = Object.fromEntries(formData);  
  
console.log(data);
```

17.40 JAVASCRIPT

RUN

```
{  
  email: "shariar@gmail.com",  
  password: "12345678"  
}
```

এটি খুব useful pattern।

17.41 Checkbox Handle করা

HTML:

17.41 HTML

MARKUP

```
<label>  
  <input type="checkbox" id="terms" />  
  
  I agree to Terms  
</label>
```

JavaScript:

17.41 JAVASCRIPT

RUN

```
const terms = document.querySelector("#terms");  
  
console.log(terms.checked);
```

Checked হলে:

TEXT

true

Unchecked হলে:

TEXT

false

17.42 Checkbox Validation

17.42 JAVASCRIPT

RUN

```
if (!terms.checked) {  
  console.log("You must accept the terms.");  
}
```

17.43 Radio Button

17.43 HTML

MARKUP

```
<label>  
  <input type="radio" name="gender" value="male" />  
  Male  
</label>  
  
<label>  
  <input type="radio" name="gender" value="female" />  
  Female  
</label>
```

JavaScript:

17.43 JAVASCRIPT

RUN

```
const selected = document.querySelector('input[name="gender"]:checked');  
  
console.log(selected?.value);
```

Male select করলে:

TEXT

male

17.44 Select Dropdown

HTML:

17.44 HTML

MARKUP

```
<select id="course">  
  <option value="">Select Course</option>  
  
  <option value="cse">CSE</option>  
  
  <option value="eee">EEE</option>  
  
  <option value="bba">BBA</option>  
</select>
```

JavaScript:

17.44 JAVASCRIPT

RUN

```
const course = document.querySelector("#course");  
  
console.log(course.value);
```

17.45 Select Validation

17.45 JAVASCRIPT

RUN

```
if (course.value === "") {  
  console.log("Please select a course.");  
}
```

17.46 File Input

HTML:

17.46 HTML

MARKUP

```
<input type="file" id="profileImage" />
```

JavaScript:

17.46 JAVASCRIPT

RUN

```
const fileInput = document.querySelector("#profileImage");  
  
console.log(fileInput.files);
```

Selected files:

17.46 JAVASCRIPT

RUN

```
console.log(fileInput.files[0]);
```

File-এর নাম:

17.46 JAVASCRIPT

RUN

```
console.log(fileInput.files[0].name);
```

17.47 File Size Check

17.47 JAVASCRIPT

RUN

```
const file = fileInput.files[0];  
  
if (file) {  
  const sizeInMB = file.size / (1024 * 1024);  
  
  if (sizeInMB > 2) {  
    console.log("File must be under 2MB.");  
  }  
}
```

17.48 Form Reset

Form-এর সব field reset করতে:

17.48 JAVASCRIPT

RUN

```
form.reset();
```

Example:

17.48 JAVASCRIPT

RUN

```
form.addEventListener("submit", function (event) {  
  event.preventDefault();  
  
  console.log("Submitted");  
  
  form.reset();  
});
```

17.49 Disable Submit Button

Duplicate submission prevent করতে button disable করা যায়।

17.49 JAVASCRIPT

RUN

```
const submitButton = form.querySelector('button[type="submit"]');

form.addEventListener("submit", function (event) {
  event.preventDefault();

  submitButton.disabled = true;

  console.log("Processing ... ");
});
```

Real-world application-এ API request চলাকালে এটি useful।

17.50 Complete Registration Form Project

এবার অনেকগুলো concept একসাথে ব্যবহার করি।

HTML

17.50 HTML

MARKUP

```
<form id="registerForm">
  <h2>Create Account</h2>

  <label for="name"> Name </label>

  <input id="name" name="name" type="text" minlength="3" required />

  <br /><br />

  <label for="email"> Email </label>

  <input id="email" name="email" type="email" required />

  <br /><br />

  <label for="password"> Password </label>

  <input id="password" name="password" type="password" minlength="8" required />

  <br /><br />

  <label for="confirmPassword"> Confirm Password </label>

  <input id="confirmPassword" type="password" required />

  <br /><br />

  <label>
    <input id="terms" type="checkbox" />

    I agree to the terms
  </label>

  <br /><br />

  <button type="submit">Register</button>
</form>
```

JavaScript

```

const form = document.querySelector("#registerForm");

const password = document.querySelector("#password");

const confirmPassword = document.querySelector("#confirmPassword");

const terms = document.querySelector("#terms");

form.addEventListener("submit", function (event) {
  event.preventDefault();

  // HTML validation
  if (!form.checkValidity()) {
    form.reportValidity();

    return;
  }

  // Password validation
  if (password.value !== confirmPassword.value) {
    confirmPassword.setCustomValidity("Passwords do not match.");

    confirmPassword.reportValidity();

    return;
  } else {
    confirmPassword.setCustomValidity("");
  }

  // Terms validation
  if (!terms.checked) {
    alert("You must accept the terms.");

    return;
  }

  // Get form data
  const formData = new FormData(form);

  const data = Object.fromEntries(formData);

  console.log(data);

  alert("Registration successful!");
});

```

17.51 এই Project-এর পুরো Flow

```

User fills form
↓
Submit
↓
preventDefault()
↓
HTML validation
↓
checkValidity()
↓
Password match?
↓
Terms accepted?
↓
FormData
↓
Object.fromEntries()
↓
API / Backend

```

এটিই real-world registration form-এর basic architecture।

17.52 Client-side Validation vs Server-side Validation

এটা খুব গুরুত্বপূর্ণ।

Client-side:

17.52 JAVASCRIPT

RUN

```
if (password.length < 8) {  
  // error  
}
```

এটি user experience-এর জন্য ভালো।

কিন্তু security-এর জন্য যথেষ্ট নয়।

কারণ user browser-এর JavaScript modify/disable করতে পারে।

তাই:

TEXT

```
Frontend Validation  
+  
Backend Validation  
=  
Proper Validation
```

Backend-এ আবার validate করতে হবে:

```
Email  
Password  
Phone  
Required fields  
Data type  
Authorization  
Business rules
```

17.53 Example: Frontend + Backend

Frontend:

17.53 JAVASCRIPT

RUN

```
const data = {  
  email: "user@gmail.com",  
  password: "12345678",  
};
```

API request:

```
POST /api/register
```

Backend:

```
Receive data  
↓  
Validate  
↓  
Sanitize/process  
↓  
Check existing user  
↓  
Hash password  
↓  
Save database
```

Frontend validation কখনোই backend validation-এর replacement নয়।

17.54 aria-invalid

Accessibility-এর জন্য invalid field indicate করা যায়।

17.54 JAVASCRIPT

RUN

```
email.setAttribute("aria-invalid", "true");
```

Valid হলে:

17.54 JAVASCRIPT

RUN

```
email.setAttribute("aria-invalid", "false");
```

17.55 Error Message দেখানোর Better Way

শুধু:

17.55 JAVASCRIPT

RUN

```
alert("Invalid email");
```

এর পরিবর্তে UI-তে message দেখানো ভালো।

HTML:

17.55 HTML

MARKUP

```
<input id="email" type="email" />

<p id="emailError"></p>
```

JavaScript:

17.55 JAVASCRIPT

RUN

```
const email = document.querySelector("#email");

const error = document.querySelector("#emailError");

if (!email.checkValidity()) {
  error.textContent = "Please enter a valid email.";
} else {
  error.textContent = "";
}
```

এতে user সরাসরি field-এর নিচে error দেখতে পাবে।

17.56 input` Event দিয়ে Live Validation

User typing করার সময় validation করতে পারি।

17.56 JAVASCRIPT

RUN

```
email.addEventListener("input", function () {
  if (email.validity.typeMismatch) {
    console.log("Invalid email");
  } else {
    console.log("Valid email");
  }
});
```

Real-life:

```
user@gmail
↓
Invalid

user@gmail.com
↓
Valid
```

17.57 Password Strength Checker

17.57 JAVASCRIPT

RUN

```
const passwordInput = document.querySelector("#password");

passwordInput.addEventListener("input", function () {
  const password = passwordInput.value;

  if (password.length < 8) {
    console.log("Weak");
  } else if (password.length < 12) {
    console.log("Medium");
  } else {
    console.log("Strong");
  }
});
```

17.58 Form Validation-এর Common Mistakes

⚠️ ❌ MISTAKE 1

শুধু frontend validation-এর উপর depend করা।

⚠️ ❌ MISTAKE 2

Password plaintext database-এ save করা।

⚠️ ❌ MISTAKE 3

User input blindly `innerHTML` -এ বসানো।

⚠️ ❌ MISTAKE 4

`trim()` না করা।

⚠️ ❌ MISTAKE 5

Duplicate form submission allow করা।

⚠️ ❌ MISTAKE 6

শুধু regex দিয়ে সব validation করার চেষ্টা করা।

MUST KNOW 🔥 17.59 Must Know — Form Architecture & Security

এই chapter থেকে অবশ্যই জানতে হবে:

```
<form>
```

```
TEXT
```

```
submit event
```

```
JAVASCRIPT
```

```
RUN
```

```
event.preventDefault();
```

```
JAVASCRIPT
```

```
RUN
```

```
input.value;
```

```
JAVASCRIPT
```

```
RUN
```

```
form.elements;
```

```
JAVASCRIPT
```

```
RUN
```

```
required;
```

```
JAVASCRIPT
```

```
RUN
```

```
minlength;  
maxlength;
```

```
JAVASCRIPT
```

```
RUN
```

```
min;  
max;
```

```
JAVASCRIPT
```

```
RUN
```

```
pattern;
```

```
JAVASCRIPT
```

```
RUN
```

```
checkValidity();
```

```
JAVASCRIPT
```

```
RUN
```

```
reportValidity();
```

```
JAVASCRIPT
```

```
RUN
```

```
validity;
```

```
JAVASCRIPT
```

```
RUN
```

```
validationMessage;
```

```
JAVASCRIPT
```

```
RUN
```

```
setCustomValidity();
```

```
JAVASCRIPT
```

```
RUN
```

```
FormData;
```

```
JAVASCRIPT
```

```
RUN
```

```
formData.get();
```

```
JAVASCRIPT
```

```
RUN
```

```
formData.set();
```

```
JAVASCRIPT
```

```
RUN
```

```
formData.append();
```

```
JAVASCRIPT
```

```
RUN
```

```
formData.delete();
```

JAVASCRIPT

RUN

```
Object.fromEntries();
```

JAVASCRIPT

RUN

```
form.reset();
```

JAVASCRIPT

RUN

```
input.checked;
```

JAVASCRIPT

RUN

```
input.files;
```

CHEAT SHEET

🧠 17.60 Quick Cheat Sheet — Form Attributes & Methods

কাজ	JavaScript
Input value	<code>input.value</code>
Checkbox	<code>input.checked</code>
File	<code>input.files</code>
Form elements	<code>form.elements</code>
Submit event	<code>"submit"</code>
Default submit বন্ধ	<code>event.preventDefault()</code>
Valid কিনা	<code>checkValidity()</code>
Browser message	<code>reportValidity()</code>
Validation info	<code>input.validity</code>
Error message	<code>validationMessage</code>
Custom error	<code>setCustomValidity()</code>
Form data	<code>new FormData(form)</code>
Data নেওয়া	<code>formData.get()</code>
Data যোগ	<code>formData.append()</code>
Data replace	<code>formData.set()</code>
Data delete	<code>formData.delete()</code>
সব data	<code>formData.entries()</code>
Form reset	<code>form.reset()</code>

Practice 1 — Login Form

তৈরি করো:

```
Email
Password
Login Button
```

Validation:

```
Email required
Valid email
Password minimum 8 characters
```

Practice 2 — Registration Form

Fields:

```
Name
Email
Phone
Password
Confirm Password
Gender
Course
Terms
```

সব validation implement করো।

Practice 3 — Password Strength

```
< 8      → Weak
8-11     → Medium
12+      → Strong
```

Practice 4 — Contact Form

Fields:

```
Name
Email
Subject
Message
```

Rules:

```
Name → required
Email → valid email
Subject → minimum 5 characters
Message → minimum 20 characters
```

Practice 5 — Student Registration

```
Student Name
Student ID
Email
Department
Semester
Phone
```

সব data `FormData` দিয়ে collect করে:

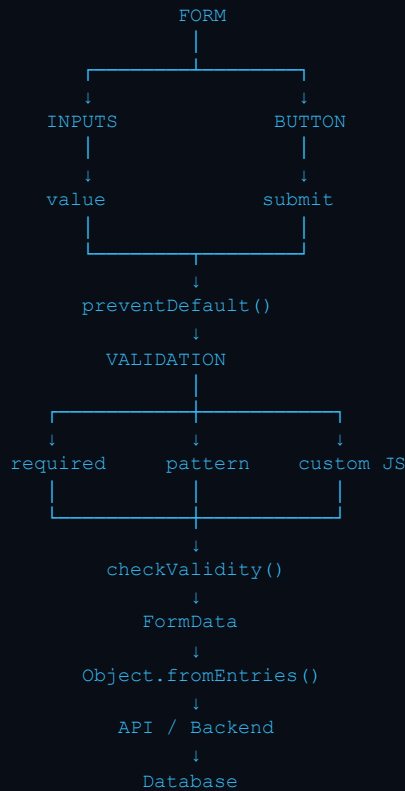
```
JAVASCRIPT
Object.fromEntries(formData);
```

RUN

এর মাধ্যমে object তৈরি করো।

SUMMARY

🎯 Chapter 17 Final Mental Map — Form Lifecycle & Validation Flow



★ ★ সবচেয়ে গুরুত্বপূর্ণ কথা

Form Handling হলো frontend development-এর অত্যন্ত গুরুত্বপূর্ণ একটি অংশ। তুমি যখন React/Next.js দিয়ে **Login, Registration, Contact Form, Dashboard, Admin Panel, Checkout, Student Portal** বানাবে—এই JavaScript fundamentals-গুলোই ভিতরে কাজ করবে।

Chapter 17 complete.

NEXT CHAPTER

📌 Chapter 18 Preview — Browser Storage & Cookies

পরবর্তী অধ্যায়ে আমরা ব্রাউজারের বিভিন্ন ক্লায়েন্ট-সাইড স্টোরেজ মেকানিজম নিয়ে বিস্তারিত শিখব—**localStorage**, **sessionStorage**, **Cookies**, **IndexedDB**, স্টোরেজ ইভেন্ট, লিমিটেশন এবং স্টোরেজ সিকিউরিটি সেবা প্র্যাকটিস।